

第9章

词典

借助数据结构来表示和组织的数字信息，可将所有数据视作一个整体统筹处理，进而提高信息访问的规范性及其处理的效率。例如，借助关键码直接查找和访问数据元素的形式，已为越来越多的数据结构所采用，这也成为现代数据结构的一个重要特征。

词典（**dictionary**）结构，即是最典型的例子。逻辑上的词典，是由一组数据构成的集合，其中各元素都是由关键码和数据项合成的词条（**entry**）。映射（**map**）结构与词典结构一样，也是词条的集合。二者的差别仅仅在于，映射要求不同词条的关键码互异，而词典则允许多个词条拥有相同的关键码^①。除了静态查找，映射和词典都支持动态更新，二者统称作符号表（**symbol table**）。实际上，“是否允许雷同关键码”应从语义层面，而非ADT接口的层面予以界定，故本章将不再过分强调二者的差异，而是笼统地称作词典，并以跳转表和散列表为例，按照“允许雷同”和“禁止雷同”的语义，分别实现其统一的接口。

尽管此处词典和映射中的数据元素，仍表示和实现为词条形式，但这一做法并非必须。与第7章和第8章的搜索树相比，符号表并不要求词条之间能够根据关键码比较大小；与稍后第10章的优先级队列相比，其查找对象亦不仅限于最大或最小的词条。在符号表的内部，甚至也不需要按照大小次序来组织数据项——即便各数据项之间的确定义有某种次序。实际上，以散列表为代表的符号表结构，将转而依据数据项的数值，直接做逻辑查找和物理定位。也就是说，对于此类结构，在作为基本数据单位的词条内部，关键码（**key**）与数值（**value**）的地位等同，二者不必加以区分。此类结构所支持的这种新的数据访问方式，即所谓的循值访问（**call-by-value**）。相对于此前各种方式，这一方式更为自然，适用范围也更广泛。

有趣的是，对这种“新的”数据访问方式，在程序设计方面已有一定基础的读者，往往会或多或少有些地抵触的倾向；而刚刚涉足这一领域的读者，却反过来会有似曾相识的亲切之感，并更乐于接受。究其原因在于，循值访问方式与我们头脑中原本对数据集合组成的理解最为接近；不幸的是，在学习C/C++之类高级程序语言的过程中，我们思考问题的出发点和方向都已逐步被这些语言所同化并强化，而一些与生俱来的直觉与思路则逐渐为我们所淡忘。比如，在孩子们的头脑中，班级的概念只不过是同伴们的一组笑脸；随着学习内容的持续深入和思维方式的反复塑化，这一概念将逐渐被一组姓名所取代；甚至可能进而被抽象为一组学号。

既已抛开大小次序的概念，采用循值访问方式的计算过程，自然不再属于CBA式算法的范畴，此前关于CBA式算法下界的结论亦不再适用，比如在9.4节我们将看到，散列式排序算法将不再服从2.7节所给的复杂度下界。一条通往高效算法的崭新大道，由此在我们面前豁然展开。

当然，为支持循值访问的方式，在符号表的内部，仍然必须强制地在数据对象的数值与其物理地址之间建立某种关联。而所谓散列，正是在兼顾空间与时间效率的前提下，讨论和研究赖以设计并实现这种关联的一般性原则、技巧与方法，这些方面也是本章的核心与重点。

^① 事实上，某些文献中所定义的词典和映射结构，可能与此约定恰好相反